

When Code Passes Once but Fails Twice: Variability and Semantic Brittleness in AI Coding Agents

Sun Chengxi
Independent Researcher
3852236135@qq.com

Abstract—AI coding agents are typically evaluated by single-run pass rates on benchmark tasks. However, a single pass does not guarantee reliable performance under repeated evaluation. We present a case study where an AI coding agent (Claude Code with deepseek-v4-flash) achieved 100% pass rate across 18 tasks in single-run evaluation, but a distributed lock bug-fixing task (H2) succeeded in only 3 out of 5 repeated runs—despite all five implementations being semantically correct. We propose a three-layer failure model that distinguishes (L1) semantic correctness, (L2) interface compatibility, and (L3) temporal stability, and introduce a variance index for quantifying run-to-run reliability. Our findings suggest that single-run evaluations may systematically overestimate the stability of AI coding agents, particularly for concurrent and distributed systems tasks.

I. Introduction

AI coding agents have achieved high pass rates on established benchmarks such as HumanEval [1] and SWE-bench [3]. These evaluations share a common methodology: each task is executed once, and the pass/fail outcome is reported as a binary result. This single-run paradigm implicitly assumes that AI code generation is deterministic—an assumption that the stochastic nature of large language models (LLMs) contradicts [4].

Our case study of Claude Code with deepseek-v4-flash on 18 software development tasks reveals a critical gap in this evaluation paradigm. While the agent achieved 100% pass rate in single-run evaluation, repeated runs of a distributed lock bug-fixing task (H2) succeeded in only 3 out of 5 attempts—a 60% pass rate with variance 2.56. This variability was not due to logical errors; all five implementations were semantically correct. Instead, failures occurred due to interface incompatibility (struct field naming mismatches causing compilation errors) and temporal instability (race conditions in concurrent goroutine execution).

We make the following contributions:

- 1) A three-layer failure model distinguishing semantic correctness, interface compatibility, and temporal stability—dimensions of AI code quality that single-run pass/fail metrics conflate.
- 2) A variance index for quantifying run-to-run reliability, supported by empirical data from 42 repeated runs across six tasks.
- 3) A detailed case study of a distributed lock task (H2) demonstrating all three failure layers.

II. Background and Motivation

A. Limitations of Single-Run Evaluation

Current benchmarks for AI coding agents rely almost exclusively on single-run pass rates. HumanEval [1] reports pass@k from independent samples but treats each sample independently, not as repeated measurements of the same task. SWE-bench [3] evaluates on real GitHub issues but does not systematically analyze run-to-run variance. The prevailing assumption is that a task either passes or fails deterministically.

This assumption is increasingly problematic. LLMs are stochastic by design—the same prompt can produce different outputs across invocations due to sampling temperature and GPU non-determinism. While prior work has examined output variability in natural language tasks, systematic analysis of variability in code generation remains limited, particularly for the agentic multi-turn coding paradigm.

B. The Need for Multi-Dimensional Evaluation

A coding agent produces not just a binary outcome, but an artifact with multiple quality dimensions:

- Does the code correctly implement the algorithm? (semantic)
- Does the code integrate with the existing codebase? (interface)
- Can the code consistently pass tests across multiple runs? (temporal)

Single-run pass rates collapse these dimensions into one bit of information, obscuring important failure modes unique to AI-generated code.

III. Three-Layer Failure Model

We propose that AI-generated code failures fall into three independent categories, forming a hierarchy of severity:

A. L1: Semantic Correctness

The algorithm correctly implements the required functionality and all test assertions pass. This is what traditional pass/fail metrics measure. An L1 failure means the code is simply wrong—the most severe and easily detected category.

B. L2: Interface Compatibility

The generated code preserves the expected API surface—function signatures, struct field names, type definitions, and interface contracts. An L2 failure occurs when semantically correct code cannot compile or integrate with the existing codebase due to surface-level mismatches. This failure mode is particularly relevant for AI-generated code: LLMs optimize for semantic equivalence but do not prioritize interface consistency across independent generations.

A human developer, by contrast, naturally preserves interface conventions because they read and understand the codebase context. The agent operates without this contextual awareness, generating valid but incompatible interfaces in different runs.

C. L3: Temporal Stability

The solution produces consistent outcomes across repeated runs under identical conditions. This layer captures non-deterministic behaviors: race conditions, timing sensitivity, scheduler-dependent outcomes, and error-handling gaps that only manifest probabilistically. L3 failures are the most insidious because they are invisible to single-run evaluation—a task can pass once and fail repeatedly without any change to the code.

Layer	Detectable by single-run evaluation?
L1: Semantic correctness	Yes
L2: Interface compatibility	Yes (occasional)
L2: Interface compatibility	No (probabilistic)
L3: Temporal stability	No

Fig. 1. Three failure layers and their detectability by single-run evaluation.

IV. Case Study: H2 Distributed Lock

A. Experimental Setup

The H2 task required fixing a distributed lock deadlock bug in Go. The correct implementation needed: (1) Lua-based atomic unlock with identity verification, (2) a watchdog goroutine for automatic TTL renewal, (3) thread-safe ownership management, and (4) proper error handling for Redis failures.

The agent was given the buggy implementation and a test suite with nine test cases covering mutex, identity verification, renewal, panic safety, concurrent locking (30 goroutines), and double-unlock safety. The experiment was repeated five times from fresh agent sessions, each starting from the same buggy code.

B. Results

TABLE I
H2 results across five independent runs.

Run	Pass/Fail	Failure Layer	Tests Passed
1	Pass	—	6/6
2	Pass	—	6/6
3	Pass	—	6/6
4	Fail	L2 (compilation)	0/6
5	Fail	L2+L3 (mixed)	4/6

C. Three-Layer Analysis

L1 — Semantic Correctness (Intact): All five implementations correctly implemented the distributed lock protocol—Lua-based atomic unlock, watchdog renewal, and ownership verification. The core locking semantics were correct in every run.

L2 — Interface Compatibility (Two Failures): Runs 4 and 5 failed initially due to struct field naming mismatches. The test suite accessed `lock2.value` directly, but the agent used field names `id` (Run 4) and `ownerID` (Run 5) instead of the expected `value`. These naming differences caused compilation errors. While the behavior was semantically correct, the test suite’s direct struct access was incompatible with the agent’s generated API surface.

L3 — Temporal Stability (Partial Failure): Run 5 additionally exhibited timing-sensitive behavior under concurrent load. The implementation discarded Lua script errors via blank identifier (`_ = script.Run(...)`), causing transient Redis failures to be silently ignored. Under the concurrent stress test (30 goroutines), scheduling variability caused the watchdog renewal to occasionally miss its deadline, leading to premature lock expiry and (occasional) failure. This failure was non-deterministic—it depended on goroutine scheduling order.

V. Variance Index

We propose a Variance Index (VI) for quantifying run-to-run reliability:

$$VI = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$$

where x_i is the number of tests passed in the i -th run and $\bar{x}$ is the mean across n runs.

A. Empirical Values

Across our variability analysis of six tasks with five runs each (30 total runs):

The five sequential tasks show $VI = 0.00$, confirming that zero variance is achievable for deterministic tasks with well-specified requirements. The H2 concurrent task shows $VI = 2.56$, reflecting the probabilistic nature of its failures.

TABLE II
Variance Index across six tasks.

Task	Type	VI
S1: Two-Sum	Sequential	0.00
S3: Binary search fix	Sequential	0.00
M1: LRU cache	Sequential	0.00
M4: SQL injection fix	Sequential	0.00
H1: Red-black tree	Sequential	0.00
H2: Distributed lock	Concurrent	2.56

B. Recommendation

We recommend that AI coding agent evaluations report:

- Single-run pass rate (current standard)
- Variance Index across 5–10 repeated runs
- Layer-wise failure classification (L1/L2/L3) for failed runs

VI. Discussion

A. Implications for Evaluation Methodology

Our findings suggest that the software engineering community should adopt multi-run evaluation as a standard practice for AI coding agents. The Variance Index provides a simple, interpretable metric that complements pass rates. For tasks with $VI > 0$, detailed failure analysis using the three-layer model can identify specific weaknesses.

B. Implications for Test Suite Design

L2 failures occurred because the test suite relied on direct struct field access rather than behavioral verification. This suggests that test suites for AI evaluation should prefer interface-based verification methods—testing behavior through public APIs rather than internal structure—to reduce false negatives from naming inconsistencies.

C. Limitations

Our variability analysis covers six of eighteen tasks. While the five sequential tasks showed zero variance, we cannot guarantee this generalizes to all deterministic tasks. Similarly, while H2 showed high variance, other concurrent tasks may behave differently. Future work should extend multi-run evaluation to larger, more diverse task sets.

VII. Conclusion

We demonstrate that single-run pass rates can overestimate the reliability of AI coding agents. Through a case study of a distributed lock task repeated five times, we identified three independent failure layers—semantic correctness, interface compatibility, and temporal stability—and observed that semantically correct code can fail at the interface or temporal layers in up to 40% of runs. The proposed three-layer model and variance index provide a more nuanced framework for evaluating AI-generated code. We recommend that future benchmarks incorporate multi-run variability analysis as a standard practice for assessing AI coding agent reliability.

References

- [1] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [2] C. E. Jimenez et al., “SWE-bench: Can language models resolve real-world GitHub issues?” arXiv:2310.06770, 2023.
- [3] T. Brown et al., “Language models are few-shot learners,” in NeurIPS, 2020.
- [4] B. Roziere et al., “Code Llama: Open foundation models for code,” arXiv:2308.12950, 2023.
- [5] R. Li et al., “StarCoder: may the source be with you!” arXiv:2305.06161, 2023.
- [6] J. Austin et al., “Program synthesis with large language models,” arXiv:2108.07732, 2021.
- [7] S. Peng et al., “The impact of AI on developer productivity: Evidence from GitHub Copilot,” arXiv:2302.06590, 2023.
- [8] D. Hendrycks et al., “Measuring coding challenge competence with APPS,” NeurIPS, 2021.